

Lab 6: API Gateway Pattern & Security

System: BadNet - Badminton Social Network **Implemented:** API Gateway (Python + Flask)

Introduction

In this lab, we implement an **API Gateway** that serves as the single entry point for all client requests to the BadNet microservices architecture. The gateway handles routing, centralized security (authentication & authorization), and error management.

This directly addresses the **Security ASR** by enforcing token validation at one central point before requests reach backend services (e.g., Match Service).

Objectives

1. Understand the role of API Gateway in microservices.
2. Implement a reverse proxy using Flask and requests.
3. Configure routing to the Match Service.
4. Implement a security stub (token validation) at the Gateway level.

Technology Stack

Tool	Purpose	Installation
Python 3.x	Core language	Already installed
Flask	Web framework for Gateway	pip install Flask
requests	Forward requests to backend services	pip install requests
Match Service	Backend service (from Lab 05)	Running on http://127.0.0.1:8001

Activity Practice 1: Project Setup and Dependencies

Step-by-step commands:

Bash

1. Create project directory

```
mkdir api_gateway
```

```
cd api_gateway
```

2. Create virtual environment

```
python -m venv venv
```

```
source venv/bin/activate
```

3. Install dependencies

```
pip install Flask requests
```

4. Create main file

```
touch gateway.py
```

File: gateway.py (Configuration)

Python

```
from flask import Flask, request, jsonify, make_response
```

```
import requests
```

```
app = Flask(__name__)
```

```
# Configuration
```

```
GATEWAY_PORT = 5000
```

```
MATCH_SERVICE_URL = 'http://127.0.0.1:8001' # BadNet Match Service
```

Activity Practice 2: Security and Routing Implementation

1. Security Functions

Python

```
def validate_token(auth_header):
```

```
    """Simulates token validation (Security Stub)"""
```

```
    if not auth_header or not auth_header.startswith("Bearer "):
```

```
        return False, "Authorization header missing or malformed"
```

```
    token = auth_header.split("Bearer ")[-1]
```

```
# Whitelist tokens
```

```
    if token in ("valid-user-token", "valid-admin-token"):
```

```
        return True, None
```

```
    else:
```

```
        return False, "Invalid or expired token"
```

```
def is_admin_token(auth_header):
    """Check if token has admin privileges"""
    return auth_header and "valid-admin-token" in auth_header
```

2. Main Routing Logic

Python

```
@app.route('/api/challenges', defaults={'path': ''}, methods=['GET', 'POST', 'PUT', 'DELETE'])
```

```
@app.route('/api/challenges/<path:path>', methods=['GET', 'POST', 'PUT', 'DELETE'])
```

```
def route_match_service(path):
    # ===== 1. SECURITY CHECK =====

    auth_header = request.headers.get('Authorization')
    is_valid, error_msg = validate_token(auth_header)

    if not is_valid:
        return jsonify({"error": "Unauthorized", "details": error_msg}), 401

    # Admin-only for write operations
    if request.method in ['POST', 'PUT', 'DELETE'] and not is_admin_token(auth_header):
        return jsonify({"error": "Forbidden", "details": "Only admins can modify challenges"}), 403

    # ===== 2. ROUTING =====

    target_url = f"{MATCH_SERVICE_URL}/api/challenges/{path}"
    if request.query_string:
        target_url += f"?{request.query_string.decode('utf-8')}"

    try:
        response = requests.request(
            method=request.method,
            url=target_url,
```

```

        headers={k: v for k, v in request.headers if k.lower() != 'host'},
        data=request.get_data(),
        timeout=5
    )

```

```

# ===== 3. RESPONSE HANDLING =====

```

```

gateway_response = make_response(response.content, response.status_code)

```

```

for key, value in response.headers.items():

```

```

    gateway_response.headers[key] = value

```

```

return gateway_response

```

```

except requests.exceptions.RequestException as e:

```

```

    return jsonify({

```

```

        "error": "Service Unavailable",

```

```

        "details": "Match Service is currently unreachable"

```

```

    }), 503

```

```

if __name__ == '__main__':

```

```

    print(f' BadNet API Gateway starting on port {GATEWAY_PORT}...')

```

```

    app.run(port=GATEWAY_PORT, debug=True)

```

Activity Practice 3: Testing & Verification

Prerequisites:

- Match Service (Lab 05) running on **port 8001**
- API Gateway running on **port 5000**

Test Cases (using cURL or Postman):

1. Test Unauthorized Access

Bash

```
curl -X GET http://127.0.0.1:5000/api/challenges
```

→ **Expected:** 401 Unauthorized

2. Test Authorized Access (User)

Bash

```
curl -X GET -H "Authorization: Bearer valid-user-token" http://127.0.0.1:5000/api/challenges
```

→ **Expected:** 200 OK + data from Match Service

3. Test Admin Operation (Forbidden for normal user)

Bash

```
curl -X POST -H "Authorization: Bearer valid-user-token" \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"challenger_id":1,"opponent_id":2,"match_date":"2026-06-01 14:00:00"}' \
```

```
http://127.0.0.1:5000/api/challenges
```

→ **Expected:** 403 Forbidden

4. Test Service Failure

- Stop Match Service → Retry authorized request → **Expected:** 503 Service Unavailable